As you'll see shortly, many factors affect the performance of multithreaded code—even something as simple as changing *which* data elements are processed by each thread (while keeping everything else identical) can have a dramatic effect on performance. So, without further ado, let's look at some of these factors, starting with the obvious one: how many processors does your target system have?

8.2.1. How many processors?

The number (and structure) of processors is the first big factor that affects the performance of a multithreaded application, and it's quite a crucial one. In some cases you do know exactly what the target hardware is and can thus design with this in mind, taking real measurements on the target system or an exact duplicate. If so, you're one of the lucky ones; in general you don't have that luxury. You might be developing on a *similar* system, but the differences can be crucial. For example, you might be developing on a dual- or quad-core system, but your customers' systems may have one multi-core processor (with any number of cores), or multiple single-core processors, or even multiple multicore processors. The behavior and performance characteristics of a concurrent program can vary considerably under such different circumstances, so you need to think carefully about what the impact may be and test things where possible.

To a first approximation, a single 16-core processor is the same as 4 quad-core processors or 16 single-core processors: in each case the system can run 16 threads concurrently. If you want to take advantage of this, your application must have at least 16 threads. If it has fewer than 16, you're leaving processor power on the table (unless the system is running other applications too, but we'll ignore that possibility for now). On the other hand, if you have more than 16 threads actually ready to run (and not blocked, waiting for something), your application will waste processor time switching between the threads, as discussed in chapter 1. When this happens, the situation is called *oversubscription*.

To allow applications to scale the number of threads in line with the number of threads the hardware can run concurrently, the C++11 Standard Thread